

ESTRATÉGIA DE EVENT SOURCING, CQRS E PROJEÇÕES — ARQUITETURA TEMPORAL E MODELO DE EXECUÇÃO — VERSÃO INICIAL

1. Objetivo do artefato

Este documento define:

- estratégia oficial de Event Sourcing;
- estratégia de CQRS;
- modelo de projeções;
- pipelines de reconstrução;
- replay operacional;
- consistência distribuída do domínio.

Seu objetivo é:

- estruturar persistência temporal;
- estabilizar fluxo causal;
- suportar reconstrução histórica;
- permitir reprocessamento determinístico;
- organizar leitura e escrita em larga escala.

Este documento representa:

- o modelo operacional interno do núcleo temporal do sistema.

2. Princípios fundamentais

2.1 O sistema é orientado a fatos históricos

O núcleo do domínio:

- persiste fatos;
- não apenas estados finais.

Estados:

- são derivados;
- reconstruíveis;
- temporais.

2.2 Leitura e escrita possuem naturezas distintas

Escrita:

- preserva causalidade;
- registra fatos.

Leitura:

- otimiza consulta;
- materializa interpretações;
- produz projeções derivadas.

2.3 O sistema deve ser reconstruível

Toda interpretação:

- deve poder ser reproduzida;
- a partir dos eventos persistidos.

2.4 Replay é requisito estrutural

Replay:

- não é ferramenta emergencial;
- é capacidade nativa do domínio.

3. Estratégia geral arquitetural

A arquitetura adota:

- Event Sourcing parcial
- CQRS parcial
- Projeções materializadas
- Consistência eventual
- Replay determinístico
- Snapshots estratégicos
- Pipelines assíncronos

4. Escopo do Event Sourcing

Contextos que utilizarão Event Sourcing

- Interpretação
- Temporalidade
- Auditoria
- Homologação
- Banco de Horas
- Reprocessamento
- Governança histórica

Contextos que NÃO exigem Event Sourcing completo

Exemplo:

- catálogo estático;
- parametrização simples;
- configurações auxiliares.

Nestes casos:

- persistência tradicional pode coexistir.

5. Estrutura do Event Store

Características obrigatórias

O Event Store deve possuir:

- Imutabilidade
- Versionamento
- Ordenação lógica
- Correlation ID
- Causation ID
- Multi-tenancy
- Replay seletivo
- Auditoria nativa

Estrutura mínima de evento

event_id
event_type
aggregate_id
tenant_id
occurred_at
captured_at
processed_at
schema_version
payload
metadata
correlation_id
causation_id

6. Estratégia de agregados

Objetivo

Agregados:

- delimitam consistência transacional.

Agregados principais previstos

- JornadaAggregate
- CompetenciaAggregate
- BancoHorasAggregate
- InconsistenciaAggregate
- HomologacaoAggregate
- RegraNormativaAggregate

Regras

Agregados:

- devem permanecer pequenos;
- evitar lock distribuído;
- evitar transações globais.

7. Estratégia de streams

Organização recomendada

Streams por:

- agregado;
- tenant;
- competência;
- domínio funcional.

Exemplos

`tenant-01.jornada.2026-05`

`tenant-01.competencia.2026-05`

`tenant-01.bancohoras.usuario-123`

8. Estratégia de snapshots

Objetivo

Reduzir:

- custo de replay completo.

Gatilhos de snapshot

Exemplo:

- quantidade de eventos
- fechamento de competência
- congelamento institucional
- checkpoints administrativos

Regras

Snapshots:

- nunca substituem eventos;
- apenas aceleram reconstrução.

9. Estratégia de replay

Objetivos do replay

Replay deve permitir:

- reconstrução histórica;
- reinterpretação normativa;
- recalculação;
- auditoria;
- restauração.

Tipos de replay

Replay completo

Reconstrói:

- todo estado do agregado.

Replay parcial

Reconstrói:

- subconjunto temporal;
- tenant específico;
- competência específica.

Replay normativo

Executa:

- interpretação sob nova regra.

Replay auditável

Preserva:

- contexto histórico original.

10. Estratégia de CQRS

Separação conceitual

Write side

Responsável por:

- consistência;
- validação;
- persistência de eventos.

Read side

Responsável por:

- consultas;
- dashboards;
- relatórios;
- visões operacionais.

11. Estratégia de projeções

Objetivo

Transformar:

- eventos históricos
em:
- modelos consultáveis.

Tipos de projeção

- Operacional
- Administrativa
- Temporal
- Analítica
- Auditorial
- Institucional

12. Projeções operacionais

Exemplos:

- saldo atual
- jornada do dia
- inconsistências abertas
- fila administrativa

13. Projeções auditoriais

Exemplos:

- árvore causal
- timeline histórica
- replay explicável
- versões interpretativas

14. Estratégia de atualização das projeções

Atualização assíncrona

Projeções:

- serão atualizadas assincronamente;
- via consumo de eventos.

Consequência

O sistema admite:

- consistência eventual controlada.

15. Estratégia de idempotência

Objetivo

Evitar:

- duplicidade;
- corrupção de projeção;
- replay inconsistente.

Regras obrigatórias

Toda projeção deve ser:

- idempotente;
- reexecutável;
- reconstruível.

16. Estratégia de versionamento de eventos

Objetivo

Permitir:

- evolução do domínio;
- compatibilidade histórica.

Regras

Eventos:

- nunca mudam semanticamente;
- novos significados exigem novo tipo/versionamento.

17. Estratégia de evolução de schemas

Compatibilidade

O sistema deve suportar:

- backward compatibility;
- replay histórico;
- múltiplas versões coexistindo.

Estratégias previstas

- upcasters
- adapters
- schema registry
- transformação incremental

18. Estratégia de consistência

Modelo adotado

Consistência eventual determinística

Características

O sistema admite:

- atraso de convergência;
- divergência transitória;
- sincronização posterior.

Mas exige:

- convergência reproduzível.

19. Estratégia de compensação

Objetivo

Corrigir:

- falhas;
- eventos inválidos;
- interpretações equivocadas.

Regra fundamental

Eventos:

- não são apagados;
- são compensados.

20. Estratégia de filas e mensageria

Requisitos

Mensageria deve suportar:

- ordenação lógica
- retry
- DLQ
- tracing
- idempotência
- replay
- particionamento

21. Estratégia de observabilidade

O sistema deve permitir

- tracing distribuído
- causalidade completa
- correlation tracking
- replay rastreável
- explicabilidade operacional

22. Estratégia multi-tenant

Regras

Eventos:

- devem possuir segregação institucional explícita.

Replay:

- nunca pode cruzar tenants.

23. Estratégia de resiliência

O sistema deve suportar

- falha parcial
- replay após desastre
- reconstrução total
- recuperação incremental
- reidratação de projeções

24. Estratégia de arquivamento

Eventos antigos

Podem ser:

- movidos para storage frio;
- preservando replay auditável.

Snapshots históricos

Devem permanecer:

- verificáveis;
- restauráveis.

25. Estratégia de segurança

Eventos críticos

Devem possuir:

- assinatura;
- rastreabilidade;
- proteção contra adulteração.

26. Estratégia de testes

Testes obrigatórios

- replay determinístico

- idempotência
- ordering
- caos distribuído
- recuperação pós-falha
- reidratação
- evolução de schema

27. Relação com bounded contexts

Cada contexto:

- possui autonomia de projeção;
- publica eventos canônicos;
- mantém consistência local.

28. Relação com temporalidade

Toda projeção:

- deve considerar vigência temporal;
- contexto normativo;
- causalidade histórica.

29. Objetivo arquitetural final

Esta estratégia busca:

- transformar histórico em ativo operacional;
- permitir reconstrução total do domínio;
- sustentar consistência temporal auditável;
- garantir evolução distribuída segura;
- preservar causalidade institucional em larga escala.